Ancestors of LLMs

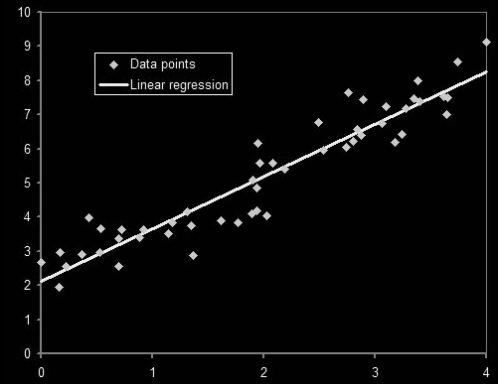
Initialize_Connection: Decades of Research... [READY]

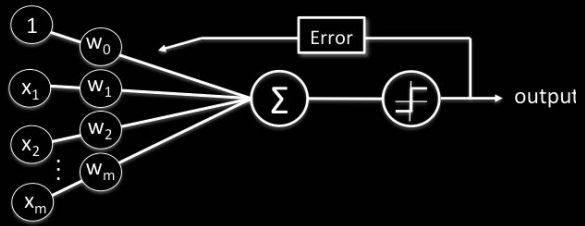
Intelligence as a Product of Research

Modern LLMs did not emerge in a vacuum. They are the refined outcome of decades spent perfecting two fundamental capabilities:

> **Representation:** How we transform the messy world of human language into mathematical vectors.
> **Recognition:** How machines learn to identify recurring patterns within those vectors.

To understand the **‘why’** behind the **‘how’**, we must look at the mathematical DNA of their ancestors.





Linear Regression: AI’s First Steps

Fundamental Machine Learning Protocol

> **Take an Input (x):** The raw data or embedding.

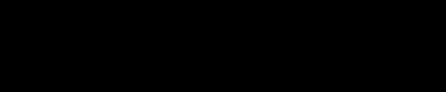
> **Apply a Weight (w):** Determining the importance of a feature.

> **Add a Bias (b):** The model’s starting point/offset.

> **Return a Prediction (z):** The raw output

From Algebra to ML

$$y = m \cdot x + b$$



$$z = w_1 x_1 + w_2 x_2 + \ldots + w_n x_n + b$$

Note: An LLM with billions of parameters is essentially a massively complex linear regression model.

Logistic Regression: The Bridge to Probability

Why Linear isn't Enough

> Human language is complex and nonlinear.

> Regression gives raw "Logits," but we need probabilities between 0 and 1.

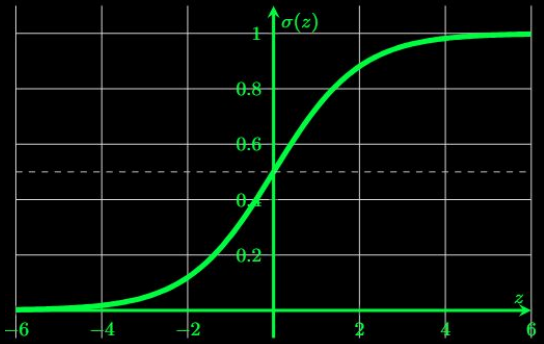
> Logistic Regression bridges this gap by "squashing" the output.

Note: Sigmoid is for **Binary Classification (yes/no)**. Because LLMs choose from thousands of words, they use **Softmax** to create a **probability map** of the entire vocabulary.

The Sigmoid Function

The Equation:

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$



Scenario Question

You are training a model to analyze student forum posts.

> **Task One:** Predict the number of words a student will type in a post.

> **Task Two:** Determine the probability (0 to 1) that the post contains a question.

What algorithms are best suited for each task?

[A] Task 1: Logistic Regression, **Task 2:** Linear Regression

[B] Task 1: Linear Regression, **Task 2:** Linear Regression with Sigmoid Activation.

[C] Task 1: Average out the number of words per post, **Task 2:** Ask ChatGPT

[D] Task 1: Sigmoid Function, **Task 2:** Logit Calculation ($z = wx+b$)

Correct Answer

[B] is Correct!

Why?

Linear Regression is used for finding relationships that result in raw numbers (e.g., counts, prices).

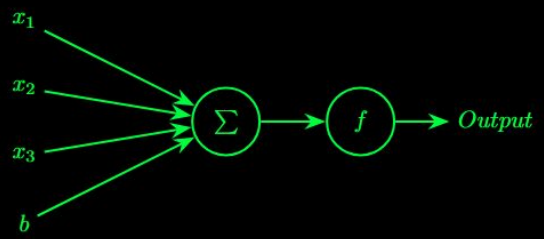
Linear Regression + Sigmoid Activation (aka **Logistic Regression**) allows for classification and probability.

The Perceptron: The First Artificial Neuron

Frank Rosenblatt’s 1958 Breakthrough

The Concept

> The **Perceptron** is the simplest form of a neural network. It was the first model designed to mimic the biological neuron’s ability to “fire” based on weighted inputs.



The Logic

> **Formula:** $a = f(\sum w_i x_i + b)$

> It combines **Linear Regression** (the weighted sum) with an **Activation Function** (the decision-maker)

> Where have we done this before?

The Impact

> In a modern **LLM**, these weights represent the strength of a connection between words. If predicting the next word after “The man is sitting on a…”, the weight for “chair” is pushed higher than for “cactus”.

Deep Learning: The Layered Architecture

If Regression is the Seed, Neural Networks are the Tree

The Input Layer

> Raw data is fed into the network.

> Text is converted into numerical vectors called **embeddings**.

Hidden Layers (thinking)

> This is where the hierarchy of features is learned.

> Layers are “hidden” because their intermediate outputs are not visible to the user.

> **Evolution:** Stacking these layers allows the model to move from recognizing simple characters to understanding complex grammar.

The Output Layer (prediction)

> The final prediction is made here.

> **In LLMs:** Typically contains a neuron for every word in its vocabulary, the one with the highest probability is generally the model’s final guess.

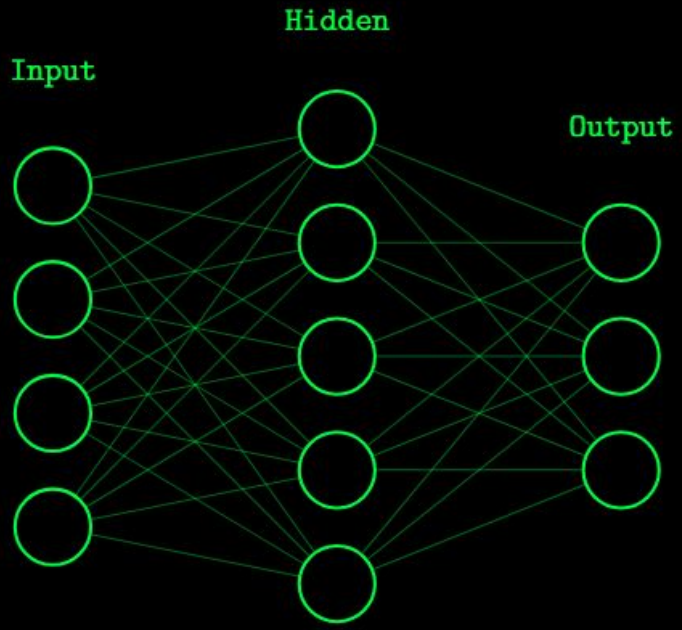
The Multi-Layer Perceptron (MLP)

Fully-Connected Dense Structure

> Every neuron in one layer is connected to every neuron in the next

> This “Dense” architecture allows the model to find subtle correlations between distant pieces of information.

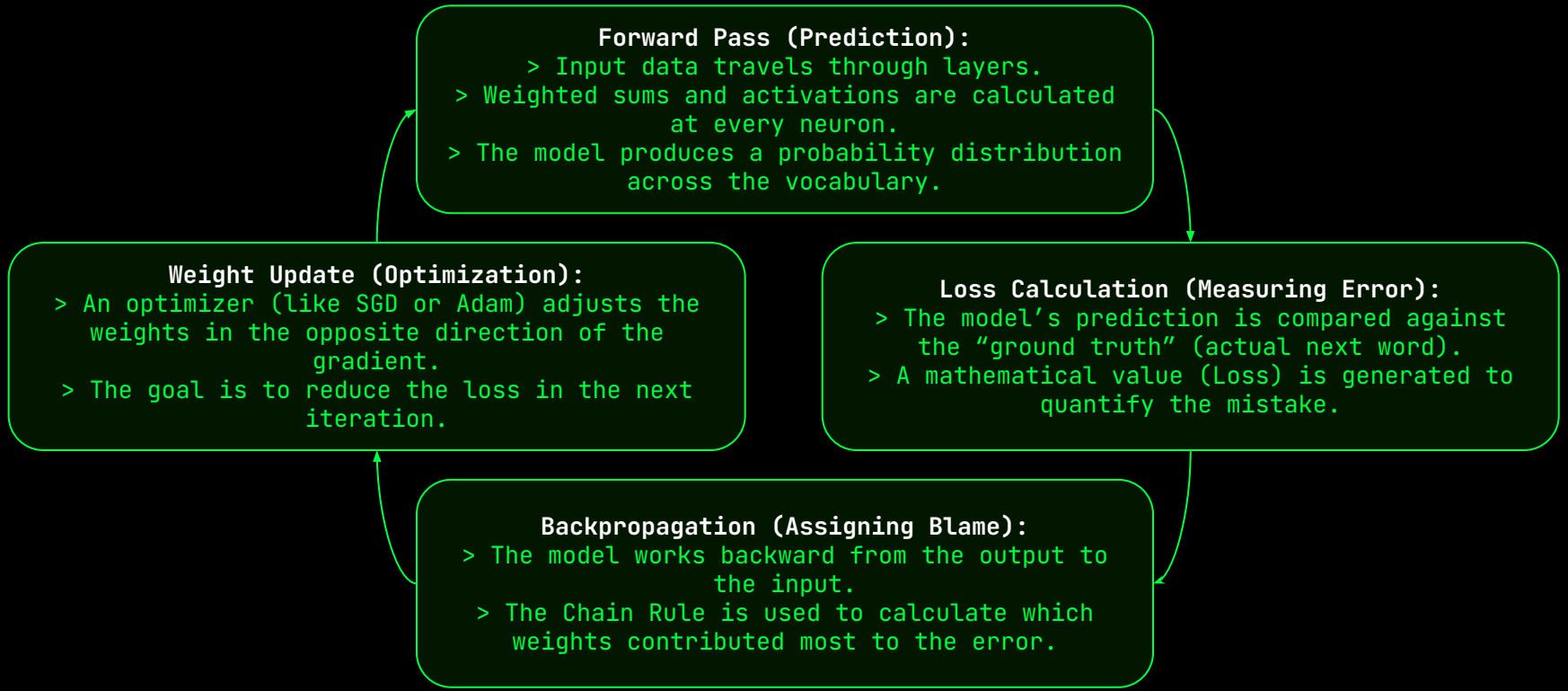
> Information flows in a single direction, known as a “feed-forward” network.



The Universal Approximation Theorem:

A feed-forward network with even a single hidden layer can represent any continuous function. This theoretical foundation is why LLMs are capable of learning any pattern in human language.

How Machines Learn: The Training Loop



The Loss Function: Defining Failure

Regression Tasks

> **Metric:** Mean Squared Error (MSE)

> **Logic:** Measure the average squared difference between the prediction (ŷ) and the actual value (y).

> **Use Case:** Predicting continuous numbers (e.g., word counts, prices)

> **Equation:**

$$MSE = \frac{1}{n}\sum(y_i - \hat{y}_i)^2$$

Classification and LLMs

> **Metric:** Cross-Entropy Loss

> **Logic:** Measures the distance between two probability distributions.

> **Use Case:** Predicting the next word in a sequence (selecting from a vocabulary)

> **Equation:**

$$L = -\sum y_i \log(\hat{y}_i)$$

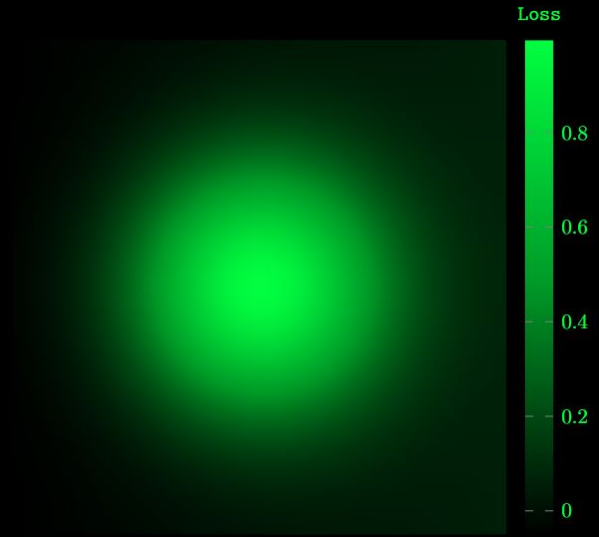
The Loss Function: Why?

Why do we use different loss functions for different tasks?

> We square the error in MSE to penalize larger outliers.

> We use logs in Cross-Entropy to penalize the model for being both **wrong** and **confident.**

> In Cross-Entropy, since y_i acts as a **binary filter** (1 for correct, 0 for wrong), the formula isolates only the probability of the true answer; if the model is confident in a wrong word, it is by definition unconfident in the correct one, triggering a massive penalty.



Cross-Entropy in Action: The Math of Uncertainty

The Mechanics of Logs

> **The Formula:** L = -log(P)

> **Logic:** We focus **only** on the probability the model assigned to the **correct** word.

> **High Probability (P ≈ 1):** log(1) = 0 The model is confident and correct; the penalty is zero (or close to it).

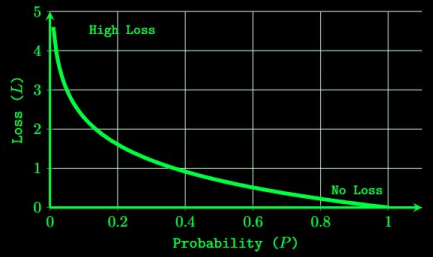
> **Low Probability (P ≈ 0):** log(0) approaches infinity. The model is severely punished for missing the answer.

The Model is Penalized for...

> **Being Wrong:** If the model predicts “Cat” when the answer was “Apple,” it is penalized.

> **Being Uncertain:** If the model splits its confidence 50/50 between two words, the loss is higher than if it were 99% sure.

> **LLM Goal:** Cross-Entropy forces the model to not just pick the right word, but to pick it with high confidence.



Backpropagation: Assigning Blame

The Chain Rule Logic

> **The Problem:** How does a tiny change in a weight at Layer 1 affect the final Loss at the Output layer?

> **The Solution:** The **Chain Rule** allows us to calculate the derivative of a complex function by multiplying the derivatives of its sub-functions.

> **The Goal:** Calculate the **Gradient** → the direction and magnitude we need to change each weight to lower the loss.

Backpropagation doesn't "fix" the model; it simply tells the model exactly how wrong each specific connection was. The **Optimizer** does the actual fixing.

Assigning "Blame"

> Each weight is given a "blame" score (the partial derivative) based on how much it contributed to the total error.

> Weights that caused the biggest mistakes receive the largest adjustments.

> **The Formula:**

$$\frac{\partial Loss}{\partial w} = \frac{\partial Loss}{\partial out} \cdot \frac{\partial out}{\partial net} \cdot \frac{\partial net}{\partial w}$$

- $\frac{\partial Loss}{\partial out}$: How much the did the neuron's final output contribute to the error?
- $\frac{\partial out}{\partial net}$: How much did the raw sum (net) change the final output?
- $\frac{\partial net}{\partial w}$: How much did that specific weight change the raw sum?

Gradient Descent: The Foggy Mountain

> **The Scenario:** You are stranded on a mountain in dense fog (high-dimensional Loss Landscape). Your goal is to reach the village at the bottom (global minimum).

> **The Gradient (The "Slope):** You can't see the village, but you can feel the slope under your feet. The gradient tells you direction is "downhill"

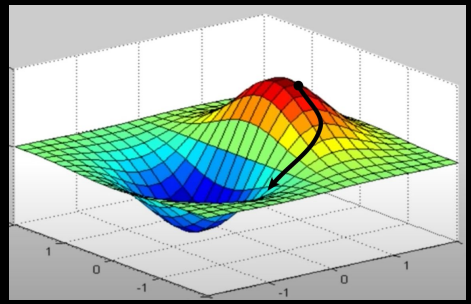
> **The Step (The Update):** You take a step in the direction of the steepest descent.

> **The Learning Rate:** This is your "Stride Length."

    > **Too Large:** You might jump over the village and land on another peak.
    > **Too Small:** It will take you forever to reach the bottom

Note: When we calculate the gradient, we take a step in the **opposite direction** because the gradient mathematically points in the direction of steepest ascent (it actually tells us how to go back up the mountain). Since our goal is to reach the bottom (the global minimum), we subtract the gradient from our current position to move in the direction of steepest ascent.

```
w_new = w_old - (learning_rate * gradient)
```



Softmax: The Final Decision

From Raw to Refined

> **The Problem (Logits):** The model's final layer outputs raw scores that can be any real number. These are impossible to interpret as "choices."

> **The Solution (Softmax):** It squashes these values into a range between 0 and 1.

> **The Goal:** Ensure the entire vocabulary sums to exactly 1.0 (100%)

> **The Formula:**

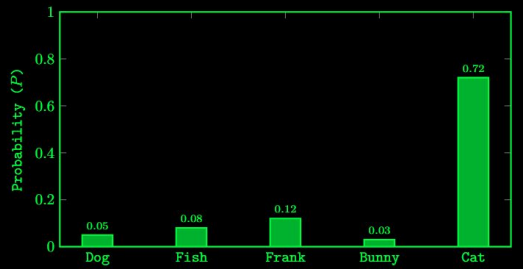
$$\sigma(z)_i = \frac{e^{z_i}}{\sum_{j=1}^{K} e^{z_j}}$$

The Winner takes Most

> **Exponential Scaling:** Because Softmax uses e^x, small differences in raw score become massive in probability.

> **Probability Map:** Every token in the model's vocabulary is assigned a percentage.

> **Normalization:** It allows us to compare "Apple" vs "Orange" on a standardized scale.



Softmax Output Distribution: Predicting the next token for "I fed cat food to my... "

The Optimization Loop: Refining the Path

SGD vs Adam

> **Stochastic Gradient Descent (SGD):** It updates weights based on one sample (or batch for mini-batch SGD). Can be noisy and “jittery” in its movement.

> **Adaptive Moment Estimation (Adam):** It uses **momentum** and adaptive learning rates for each weight. It’s like a hiker with a memory of the previous slope.

> **Why Adam?** It handles sparse gradients and noisy data much better, making it the industry standard for LLMs.

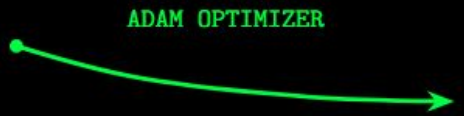
The Power of Mini-Batches

> **Batch:** Calculating the gradient for the *entire* dataset (Too slow/memory intensive)

> **Mini-Batch:** Breaking the data into small chunks (e.g., 32, 64 samples).

> **Stability:** Averages out the noise of individual outliers, providing a “smoother” gradient signal.

> **Efficiency:** Allows for parallel processing on GPUs.



Recurrent Neural Networks: The First Memory

The Problem of “Now”

> **Standard Networks:** Treat every input (word) as an isolated event. They have no “memory” of what came before.

> **The Sequential Challenge:** In language, the meaning of a word depends entirely on the words preceding it.

> **RNN Solution:** Introducing a **Loop**. The network processes the current input **and** its own previous output.

The Hidden State (h_t)

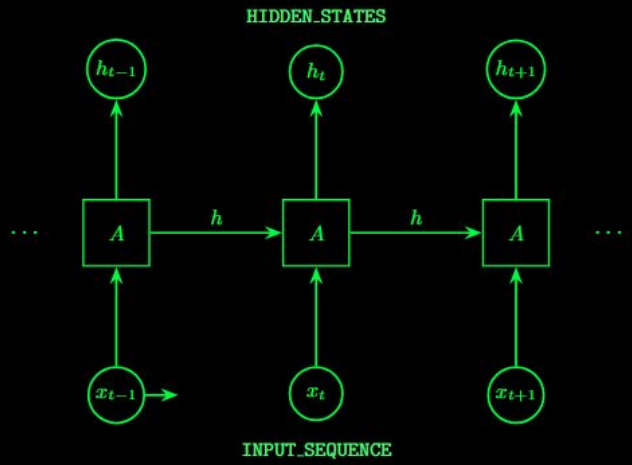
> **The “Memory”:** At each step, the RNN updates a “Hidden State.” A mathematical vector that carries context forward.

> **Persistence:** h_t is a function of both the current word (x_t) and the previous hidden state (h_{t-1}).

> **Formula:** $$h_t = \tanh(W_{hh}h_{t-1} + W_{xh}x_t)$$

> **The Result:** The model can theoretically “remember” the beginning of a sentence while reading the end.

Context is Everything.



The First Major Step

> **Why care about memory?** Language is not a static set of points, but a trajectory. By using a **Hidden State** we allow the model to maintain a "mental summary" of the past.

> Without a Hidden State, a word like "Bank" is just a coordinate. With it, the model knows if "bank" refers to a river or a financial institution based on the **context**.

> LLMs don't just predict words; they navigate the **intent** behind a sequence.

> RNNs were the first major step towards the LLMs we use today: moving from a simple classification to a deep understanding of **context** over time.

The Vanishing Gradient Problem: Fading Memory

The Long-Range Struggle

> **The Concept:** In theory, RNNs can remember everything. In practice, they usually “forget” the beginning of a sentence by the time they reach the end.

> **The Cause:** As we backpropagate through time, we multiple gradients by the same weight matrix repeatedly.

> **The Effect:** If the weights are small (<1), the gradient shrinks exponentially until it becomes 0.

Why it Breaks Models

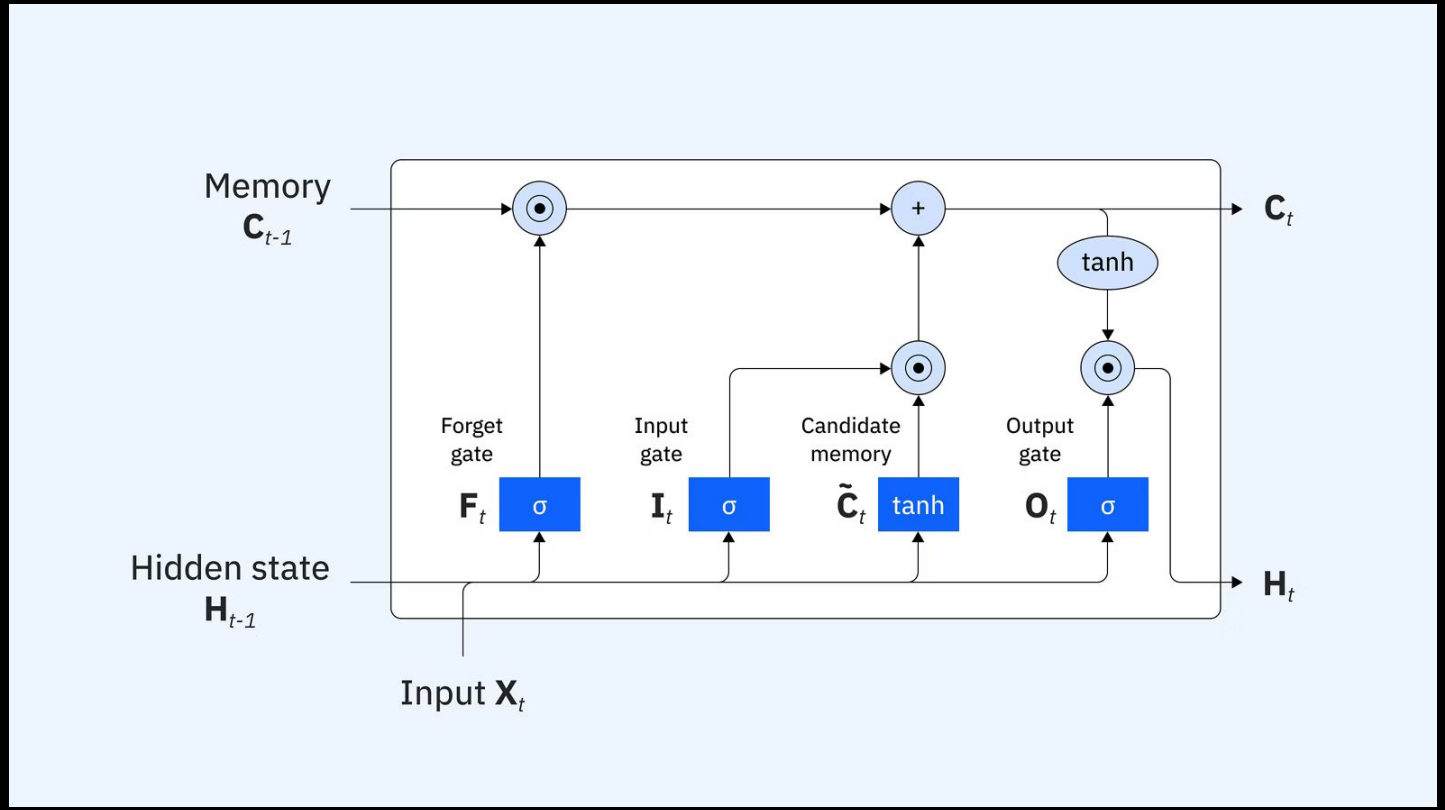
> **Loss of Context:** If the gradient vanishes, the model can’t learn the relationship between a word at index 0 and a word at index 50.

> **Broken Dependencies:** It might forget the subject of a sentence, leading to grammatical or logical “hallucinations.”

Let’s Think:

> Imagine all the gradients are 0.1
> We multiply that repeatedly as we backpropagate:
> > 0.1 * 0.1 * 0.1 ... * 0.1
> The final value will shrink exponentially

LSTMs: The Architecture of Long-Term Memory



Scenario Question

The Scenario:

> You are training a standard LSTM to generate automated code documentation. You notice that in very long sentences (100+ words), the model starts correctly, but eventually fails to maintain subject-verb agreement.

> **Example:** “The **code** that the engineers from the satellite division wrote during the hackathon... **were** buggy.”

The Challenge:

> **The “Distractor” Effect:** Why did the model predict “**were**” (plural) when the subject “**code**” is singular?

> **The Gate Audit:** Which specific LSTM fat was responsible for “cleaning’ the hidden state so thoroughly that the original subject was forgotten?

> **The Structural Limit:** Even though LSTMs have a “cell state” highway to solve vanishing gradients, why does this specific error still happen in sequential models?

Correct Answers

> The Error: Recency Bias
> The plural noun “**engineers**” is much closer to the verb than the singular object “**code**”

> The Gate: The Forget Gate
> Over the 10+ steps between the subject and the verb. The **Forget Gate** progressively diminished the “singular” feature to make room for newer information.

> The Bottleneck: Fixed-Length Vector
> Sequential models must compress the entire past into a single, fixed-size vector. Eventually, the most recent tokens “crowd out” the distant ones, regardless of their importance.

Conclusion: From Architectures to Numbers

The Summary of Ancestors

> **Neural Networks** provide the “brain” that adjusts weights via Gradient Descent.

> **RNNs/LSTMs** added the “memory” (Hidden State) to handle sequences.

> **The Limitation:** Even though LSTMs solved the Vanishing Gradient Problem, their **sequential nature** creates an unavoidable computational bottleneck that prevents massive GPU parallelization and still suffers from a “recency bias” that fails to capture the precise, global relationships required for the scale of modern LLMs.

NEXT LECTURE: We move from the “Loop” to the “Map.” We will explore **Probabilistic Language Models**, N-Grams, and how we turn a dictionary into a coordinate system.

FUTURE PREVIEW: Eventually, we will learn how to destroy the sequential bottleneck entirely. No more loops → just **Self-Attention** and pure parallel power.